Smart IT Helpdesk — Project Proposal

AI Capstone Project (Module 27)

Sheikh Mohammad Saadi October 2026

1. Problem statement

In many organisations, employees report IT problems through phone calls, chat messages, email and hallway conversations. The IT team then has no single record of who reported what, which issues are still open, or how a similar problem was solved last time. Three concrete problems follow:

- **Lost context and slow triage.** Requests arrive without a consistent category, so an IT officer has to read every message and decide where it belongs before work can start.
- **No visibility for employees.** People cannot see whether their issue was received or what its status is, so they follow up repeatedly.
- **Repeated questions.** Many requests (“How do I reset my password?”, “Wi-Fi keeps disconnecting”) already have a documented answer, but employees cannot find it.

The problem is real and familiar: it is drawn from the author’s experience managing IT support in a multi-unit industrial group in Dhaka.

2. Objectives

1. Give every IT issue a ticket with an owner, status, priority, conversation and audit history.
2. Use a machine-learning model to **suggest a ticket category** from free-text descriptions, while the person keeps the final decision.
3. Let administrators upload IT guides (PDF/TXT) and let employees **ask questions answered with quoted passages from those guides**, with source links.
4. Measure the AI component honestly, with a held-out test set, macro-F1 and a confusion matrix.
5. Deploy the system publicly on free hosting, with the code, history and documentation on GitHub.

3. Scope

In scope: employee and administrator accounts; ticket creation, search, filters, replies and status workflow; six categories (Email, Network, Hardware, Software, Printer, Access); AI category suggestion with abstention; PDF/TXT knowledge library; private source-backed chat; responsive web UI; deployment on Render and Neon.

Out of scope: automatic repairs or access grants; email/SMS notifications; OCR of scanned documents; validated Bangla/Banglish support; generative LLM answers.

4. Proposed solution

Smart IT Helpdesk is a web application with three layers:

Layer	Responsibility
Frontend	React single-page app: sign-in, dashboard, tickets, AI suggestion panel, knowledge library, Ask helpdesk chat, AI insights
Backend	FastAPI REST API: authentication, permissions, ticket workflow, AI endpoints, document ingestion and retrieval
Database	PostgreSQL: users, sessions, tickets, replies, history, AI assessments, documents, passages and conversations

5. AI approach

(a) Ticket category classifier. Ticket text is converted into word TF-IDF (1–2-grams) and character TF-IDF (3–5-grams) features and classified with Logistic Regression (scikit-learn). This model is fast, interpretable, runs on a 512 MB free server and needs no paid API. The data are split by *intent group* into training, validation and test sets so that paraphrases of the same issue cannot leak between sets. The validation set selects a confidence threshold; below it the model **abstains** and the user chooses manually.

(b) Knowledge retrieval. Uploaded guides are split into passages and indexed with TF-IDF. A question is matched by cosine similarity, and the top passages are returned verbatim with document and page references. When nothing matches well enough, the system says so rather than inventing an answer.

Evaluation plan: accuracy, macro-F1, per-class precision/recall, confusion matrix, suggestion coverage and accuracy of

accepted suggestions, compared with a majority-class baseline, plus unrelated “negative control” requests.

6. Technology stack

Area	Technology
Frontend	React, Vite, TypeScript, Tailwind CSS
Backend	Python, FastAPI, Pydantic, SQLAlchemy 2.0, Alembic
Database	PostgreSQL (Neon Free in production)
AI / ML	scikit-learn (TF-IDF, Logistic Regression), pypdf
Security	Argon2 password hashing, HttpOnly server-side sessions, origin checks
Deployment	Docker on Render Free (single service serving API and UI)
Version control	Git and GitHub, one branch and pull request per phase

7. Plan

Phase	Deliverable
1–2	Project skeleton, accounts, PostgreSQL tickets and admin workflow
3	AI category classifier, evaluation and AI insights page
4	Knowledge upload and source-backed chat
5	Docker, Render and Neon deployment
6	Final documentation, screenshots and report

Expected impact: faster triage, fewer repeated questions, and a measurable record of IT support work — with a design that keeps people in control of every AI suggestion.